

TREES

Binary Trees

A binary tree is a finite set of elements that either empty or partitioned into three disjoint subsets.

The first subset contains a single element called root of the tree. The other two subsets are themselves binary trees, called left and right subtrees of the original tree. A left or right subtree can be empty. Each element of a binary tree is called a node of the tree.

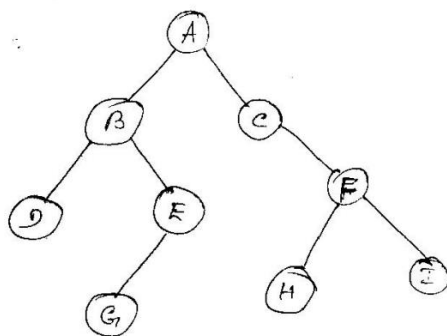


fig: binary tree

The above binary tree consists of 9 nodes with A as root. If left subtree is rooted at B and right subtree is rooted at C. This is indicated by the two branches emanating from A to B on the left and A to C on the right.

The absence of a branch indicates an empty subtree.

For example, the left subtree of the binary tree rooted at C and the right subtree of the binary tree rooted at E are both empty. The binary trees rooted at D, G, H and I have empty right and left subtrees.

The following figures are not binary trees (fig (a) and fig (b))

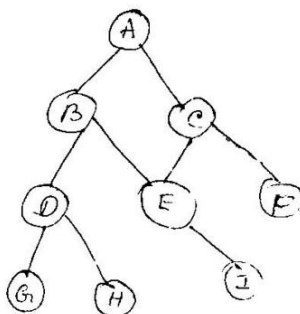


fig (a)

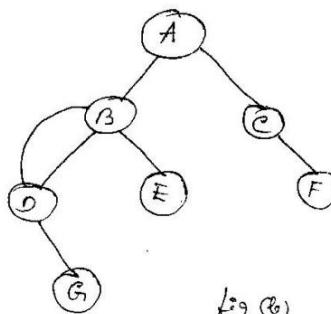


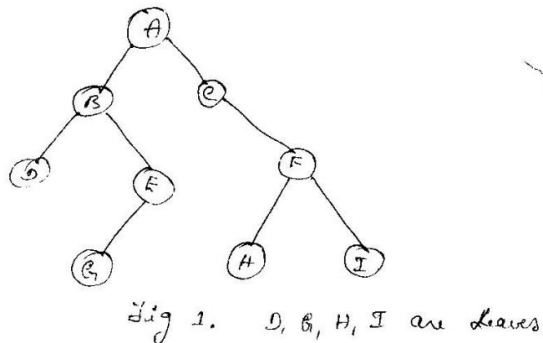
fig (b)

Some definitions:

If A is the root of a binary tree and B is the root of its left or right subtree, then A is said to be the father of B or parent of B and B is said to be the left son(child) or Right son(child) of A.

A node that has no child nodes is called a leaf.

Example:



Node N1, is an ancestor of Node N2 (ie, N2 is a descendent of Node N1) if N1 is the either parent of N2 or parent of some ancestor of N2.

For example, in the above example, (Fig1), A is the ancestor of G. H is descendent of C.

E is neither an ancestor nor descendent of C.

A Node N2 is a left descendent of Node N1 if N2 is either the left child of N1 or a descendent of the left child of N1.

Example, In fig1, D and E are left descendent of A.

A Node N2 is a right descendent of Node N1 if N2 is either the right child of N1 or a descendent of the right child of N1.

Example, In fig1, H and I are right descendent of A.

Two nodes are brothers or siblings if they are left and right child nodes of same parent.

Going from the root to the leaves is called descending the tree.

Going from the leaves to the root is called climbing the tree.

If every non leaf node in a binary tree has non empty left and right subtrees, then such type of tree is termed as “strictly binary tree”.

The fig1 is not a strictly binary tree because node C and E have one child node each.

A strictly binary tree is shown below:

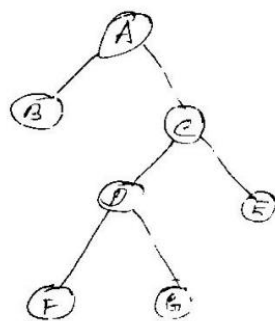


Fig 2 (strictly binary tree)

A Strictly binary tree with n leaves always contains $2n-1$ nodes.

The level of a node in a binary tree is defined as follows:

The root of the tree has level 0, and the level of any other node in the tree is one more the level of its parent.

For example, in fig 1, the level of node E is 2 and node H is at level 3.

The depth of a binary tree is the maximum level of any leaf in the tree. This equals the length of the longest path from the root to any leaf.

For example, in fig1, the depth is 3.

A complete binary tree of depth d is the strictly binary tree and all of whose leaves are at level d .

For example, the following tree is the complete binary tree.

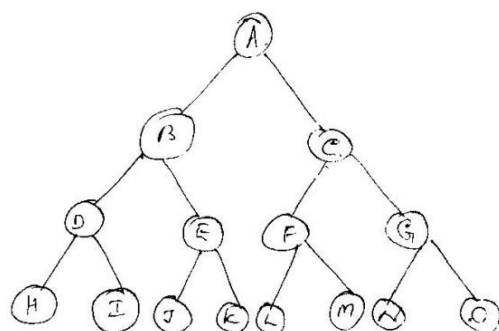
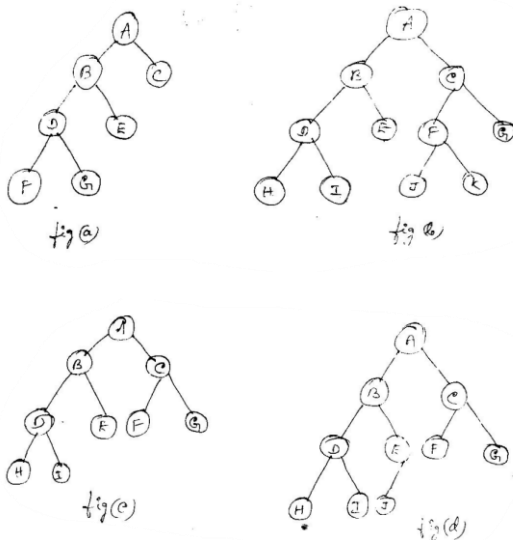


Fig 3. Complete binary tree.

A binary tree of depth s is almost complete binary tree if:

1. Any node n_d , at level less than $d-1$ has two sons.
2. For any node n_d in the tree with a right descendent at level d , n_d must have a left child and every left descendent is either a leaf at level d or has two child nodes.

Example:



The strictly binary tree of fig (a) is not almost complete binary tree since it violates condition 1.

The strictly binary tree of fig (b) is not almost complete binary tree since it violates condition 2. This satisfies condition 1. But it has a right descendent at level 3 (J) and left descendent at level 2 (E).

The strictly binary tree of fig (c) is an almost complete binary tree.

The binary tree of fig (d) is an almost complete binary tree.

Operations on Binary Tree:

The Primitive Operations that can be applied to a binary tree are listed as below:

info(p) – indicates the contents of node n_d , where p is a pointer to a node n_d .

left(p) – indicates pointer to the left child of node n_d .

right(p) – indicates pointer to the right child of node n_d .

Applications of Binary Tree:

A Binary Tree is a useful data structure when two way decisions must be made at each point in a process.

For example, suppose that we wanted to find all the duplicates in a list of numbers. One way of doing this is to compare each number with all those that precede it. However, this involves a large number of comparisons.

The number of comparisons can be reduced by using a binary tree. The first number in the list is placed in a node that is established as root of a binary tree with empty left and right subtrees. Each successive numbers in the list is then compared to the number in the root. If it matches, we have a duplicate. If it is smaller, we examine the left subtree; if it is larger, we examine the right subtree. If the subtree is empty, the number is not a duplicate and is placed at that position in the tree. If the subtree is non empty, we compare the number to the contents of the root of the subtree and the entire process is repeated with the subtree.

The algorithm to insert a number into a binary tree and to check if there is duplicate:

Start:

```
scanf("%d", &number); /* read the first number and insert it into the root node of the binary tree */
tree = maketree(number);
while(there are numbers left in the input)
{
scanf ("%d", &number);
p=q=tree;
while(number!=info(p) && q!=NULL)
{
p=q;
if(number<info(p))
    q=left(p);
else
    q=right(p);
} /* end of inner while loop*/
if(number == info(p))
printf("%d is duplicate\n", number);
```

```

else if(number<info(p))
    setleft(p, number);
    else setright(p, number);
} /* end of outer while */

```

Note:

maketree(number) – Creates a new binary tree consisting of a single node with information field 'number' and returns a pointer to that node.

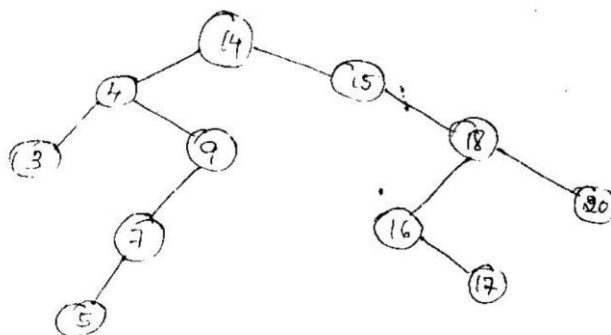
setleft(p, number) – Accepts a pointer p to a binary tree node with no left child. It creates a new left child of node p with information filed number.

setright(p, number) – Accepts a pointer p to a binary tree node with no right child. It creates a new right child of node p with information filed number.

Create a binary tree for the numbers given below:

14, 15, 4, 9, 7, 18, 3, 5, 16, 4, 20, 17, 9, 14, 5.

Constructed Tree:



A **binary search tree** has the property that all the elements in the left subtree of a node n are less than the contents of n and all the elements in the right subtree of n are greater than or equal to the contents of n .